

# Brain 'O' Clock

Cognitive Alarm Platform

Application Workflow, Wireframes & Frontend Screen Documentation

---

Covers: App Workflow • Onboarding Wireframes • App.tsx • SplashScreen.tsx • LoginScreen.tsx •  
SignupScreen.tsx

React Native (TypeScript) Frontend • FastAPI Backend

# Introduction

---

The Cognitive Alarm Platform is an intelligent alarm application designed to help users wake up on time by combining traditional alarm functionality with cognitive challenges. Unlike conventional alarm apps that allow users to dismiss alarms with a single tap, this application requires users to complete a mental challenge before the alarm can be turned off. The primary goal is to improve wakefulness, reduce oversleeping, and encourage healthy daily routines.

The application follows a simple, user-friendly workflow consisting of three major phases: **Onboarding**, **Core Alarm Loop**, and **Habit Review & Settings**. Each phase contributes to delivering a seamless experience while ensuring the alarm functions even without an internet connection.

## Document Contents

- Section A — Application Workflow (Onboarding, Alarm Loop, Habit Review & Settings)
- Section B — Wireframe Reference Screens
- Section C — Frontend Screen Logic: App.tsx, SplashScreen.tsx, LoginScreen.tsx, SignupScreen.tsx

## Technology Stack

**Backend:** FastAPI, SQLAlchemy 2.0, Alembic, Pydantic v2, Docker, GitHub Actions, Sentry, Prometheus/Grafana.

**Frontend:** React Native, TypeScript, Zustand (state management), React Query (server-state/caching), WatermelonDB/Realm (offline-first local database), Notifee (local notifications/alarm firing), EAS & Fastlane (build and release pipeline).

# Section A — Application Workflow

---

## 1. Onboarding Phase

The workflow begins when a user launches the application for the first time.

### Splash Screen

The application displays the Brain O' Clock splash screen, presenting the app logo and branding while the system initializes required services. During this stage, the application checks whether the user has previously logged in and loads essential local data.

### Login Screen

Returning users authenticate using their registered email and password. Additional login options such as Google and Apple Sign-In simplify authentication. Features like “Remember Me” and “Forgot Password” improve usability.

### Sign-Up Screen

New users create an account by entering their basic details. They can also register using their Google or Apple account for faster onboarding.

Once authentication is successful, the user enters the main application.

## 2. Home Screen (Alarm Dashboard)

---

After logging in, users reach the Home Screen, which serves as the application's dashboard.

### The dashboard displays:

- Existing alarms
- Alarm activation status
- Alarm labels
- Quick enable/disable switches
- Floating Action Button (+) for creating new alarms

### Users can:

- Create a new alarm
- Edit an existing alarm
- Delete alarms
- Enable or disable alarms

The application stores alarms locally to ensure they continue functioning without an internet connection.

## 3. Alarm Creation Workflow

---

When the user taps the “+” button, the Add/Edit Alarm screen opens. Here the user configures:

- Alarm time
- Repeat days
- Alarm label
- Challenge type
- Challenge difficulty

After entering the required details, the alarm is saved to the device's local storage. The operating system schedules the alarm using native background scheduling services, ensuring it triggers even if the application is closed, the device has no internet, or the application is running in the background.

## 4. Alarm Trigger Workflow

---

At the scheduled time, the application automatically launches the Alarm Firing screen. This screen displays the current alarm, the ringing sound, a Snooze button, and a Dismiss button.

However, unlike a traditional alarm application, pressing “Dismiss” does not immediately stop the alarm. Instead, the application transitions to the Challenge Screen.

## 5. Cognitive Challenge Workflow

---

The Challenge Screen presents a randomly generated mental exercise selected according to the user's chosen challenge type and difficulty. Possible challenge categories include mathematical calculations, memory challenges, pattern recognition, logical reasoning, and other cognitive exercises — for example,  $7 \times 8 = ?$  with multiple-choice options displayed. The application continuously validates the user's response.

### Correct Answer

- Alarm stops
- Success screen is displayed
- Wake-up statistics are updated
- Completion is stored in the user's history

### Incorrect Answer

- Alarm continues ringing
- A new challenge of the same difficulty is generated
- User must solve another challenge

This process repeats until the correct answer is provided, ensuring the user is mentally alert before dismissing the alarm.

## 6. Success / Retry Screen

---

Once the challenge is successfully completed, the application displays the Success screen. The screen confirms successful alarm completion and provides a Continue button. If the user previously answered incorrectly, the Retry workflow automatically generated another challenge before eventually reaching this screen. Selecting Continue returns the user to the Home Screen.

## 7. Habit Review Dashboard

---

The application records every completed alarm. The Habit Dashboard presents performance metrics such as:

- Alarm completion rate
- Daily and weekly wake-up history
- Streak count
- Success percentage
- Habit progress visualization

These insights help users monitor consistency and build healthier waking habits over time.

## 8. Profile Management

---

Users can manage personal preferences through the Profile page. Available options include:

- Wake-up goals
- Sleep schedule
- Preferred challenge difficulty
- Personal account information
- Logout

These preferences personalize the alarm experience according to individual routines.

## 9. Application Settings

---

The Settings page allows users to configure application-wide options:

- Push notifications
- Bedtime reminders
- Privacy settings
- Account management
- Help and support

All preferences are stored locally and synchronized when internet connectivity becomes available.

### Overall Workflow Summary

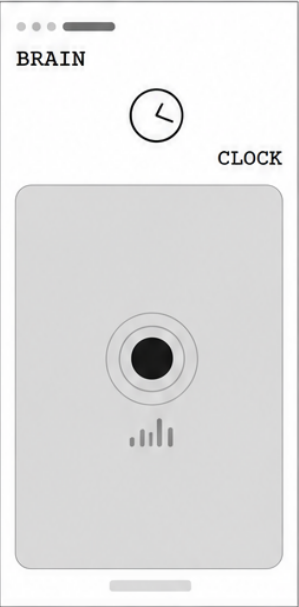
- Launch App → Splash Screen → Login / Sign Up → Home Dashboard
- Home Dashboard → Create or Edit Alarm → Save Alarm → Scheduled Time Reached
- Scheduled Time Reached → Alarm Rings → Challenge Screen
- Challenge Screen → Correct → Success → Continue → Home Dashboard
- Challenge Screen → Incorrect → New Challenge → back to Challenge Screen
- Home Dashboard → Habit Dashboard & Statistics

# Section B — Wireframe Reference

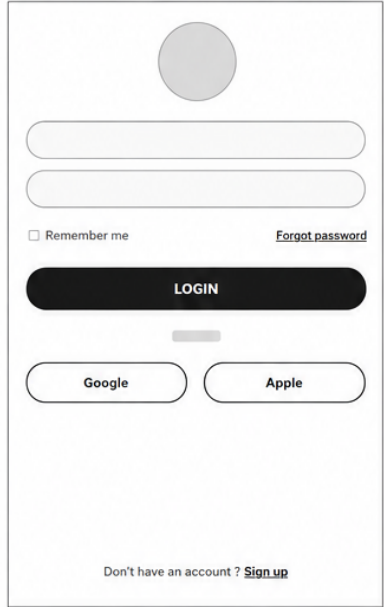
## Onboarding & Core Alarm Loop

### ONBOARDING

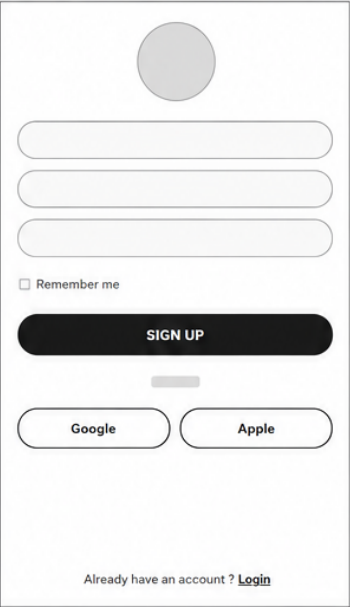
1. Splash — Brain 'O' Clock



2. Login



3. Sign Up

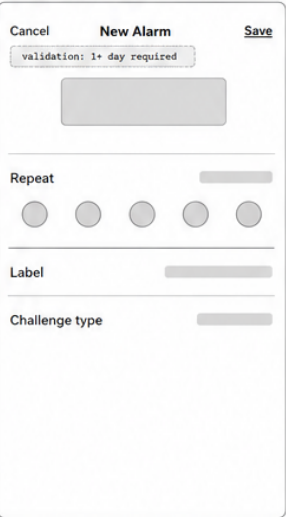


### CORE ALARM LOOP

4. Home / Alarm List



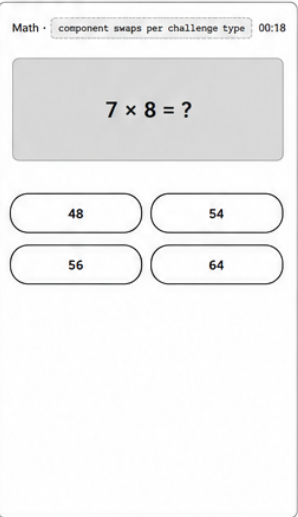
5. Add / Edit Alarm



6. Alarm Firing



7. Challenge

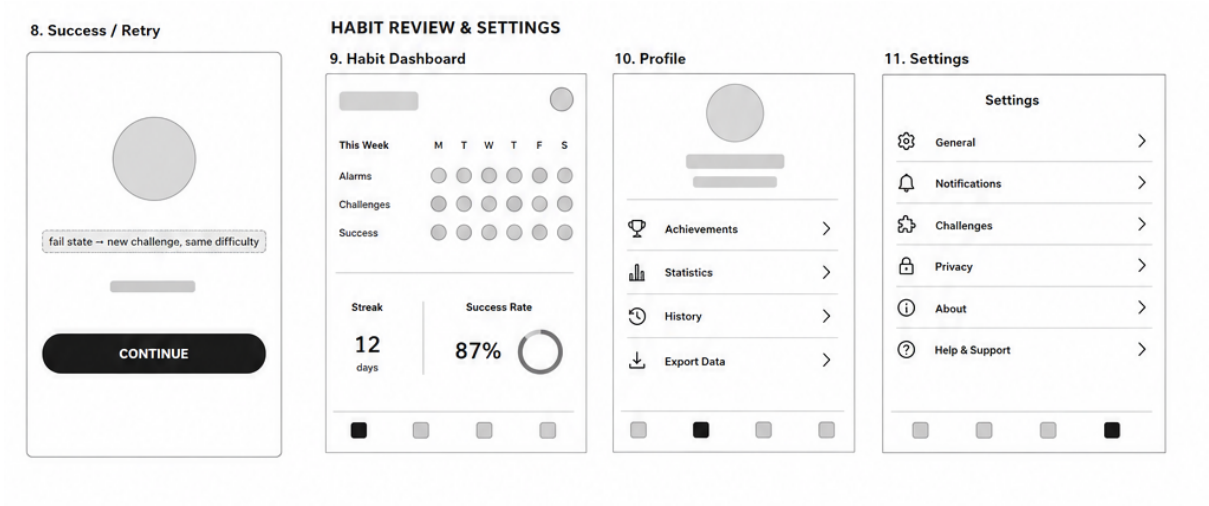


Screens 1–3: Splash, Login, Sign Up. Screens 4–7: Home/Alarm List, Add/Edit Alarm, Alarm Firing, Challenge.



# Wireframe Reference (continued)

## Success / Retry & Habit Review & Settings



Screens 8–11: Success/Retry, Habit Dashboard, Profile, Settings.

## Section C — Frontend Screen Logic

---

### App.tsx — Root Application Component

App.tsx is the entry point of the React Native application. It does not render any screen content itself; instead it wires together the providers, navigation, and global services that every screen depends on.

### Responsibilities

- **Provider composition** — wraps the component tree with the React Query `QueryClientProvider` (server-state caching/sync), the Zustand store initializer, and a `SafeAreaProvider/GestureHandlerRootView` for React Native.
- **Navigation container** — mounts React Navigation's `NavigationContainer` and defines the root stack/switch navigator that decides whether to show the Auth stack (Splash → Login → Sign Up) or the Main stack (Home, Alarm, Habit, Profile).
- **Local database bootstrap** — initializes the offline-first WatermelonDB/Realm database instance once at startup so every screen can query alarms/habit data even when offline.
- **Notification setup** — configures Notifee channels and background/foreground event listeners so alarm notifications can fire and launch the Alarm Firing screen even when the app was killed.
- **Error & crash reporting** — initializes Sentry so runtime errors on device are captured centrally.

### Simplified Structure

```
export default function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <SafeAreaProvider>
        <NavigationContainer>
          <RootNavigator />
        </NavigationContainer>
      </SafeAreaProvider>
    </QueryClientProvider>
  );
}
```

In effect, App.tsx acts as the “composition root”: it does not know about individual screen UI, only about which global services must exist before any screen can render.

# SplashScreen.tsx — Startup & Auth Check

---

SplashScreen.tsx is the first screen rendered inside the Auth stack. It matches the wireframe's dark-themed splash design: the app title with a live-ticking analog clock replacing the letter “O,” a bell/rings hero illustration, and a loading indicator.

## What it does

- **Live clock animation** — uses a `useEffect` with `setInterval` (or `react-native-reanimated`) to update the analog clock hands every second, giving the “O” its ticking-clock appearance.
- **Session check** — on mount, it reads a persisted auth token (`SecureStore/AsyncStorage` or the Zustand auth slice) to determine whether the user is already logged in.
- **Local data warm-up** — opens the `WatermelonDB/Realm` database connection and hydrates the Zustand store with cached alarms so the Home screen has data ready immediately, even offline.
- **Navigation redirect** — after the check resolves (typically wrapped in a short minimum-duration timer so the branding is visible), it calls `navigation.replace()` to either the Login screen (no valid session) or the Home dashboard (valid session), so the user can never navigate back to the splash screen.

## Simplified Structure

```
useEffect(() => {
  const clockTimer = setInterval(tickClock, 1000);
  (async () => {
    await initDatabase();
    const token = await getStoredToken();
    navigation.replace(token ? 'Home' : 'Login');
  })();
  return () => clearInterval(clockTimer);
}, []);
```

Because it uses `replace` rather than `navigate`, the splash screen is removed from the navigation stack entirely once the redirect happens.

# LoginScreen.tsx — Returning User Authentication

---

LoginScreen.tsx renders the dark-themed form shown in the wireframe: avatar placeholder, email input, password input, a “Remember me” checkbox, a “Forgot password” link, the primary LOGIN button, and Google/Apple social buttons.

## What it does

- **Form state** — manages email/password fields with controlled `TextInput` components (or `React Hook Form`) and inline validation (valid email format, non-empty password).
- **Submit handler** — calls a `React Query` mutation (`useLoginMutation`) that POSTs credentials to the `FastAPI /auth/login` endpoint.
- **Token persistence** — on success, stores the returned JWT/session token in `SecureStore` and updates the `Zustand` auth slice, then navigates to the Home dashboard.
- **Remember me** — when checked, persists the token beyond the current session; when unchecked, the token is kept in memory only.
- **Social sign-in** — the Google/Apple buttons trigger native OAuth flows (`expo-auth-session` or the native Apple/Google SDKs) and exchange the resulting identity token with the backend for a session, reusing the same success handler as email/password login.
- **Forgot password** — navigates to a password-reset flow that emails a reset link/code.
- **Error feedback** — invalid credentials or network failures surface as inline error text near the form rather than a blocking alert, matching the offline-first, non-disruptive design.

The “Don't have an account? Sign up” link at the bottom navigates to `SignupScreen.tsx`.

# SignupScreen.tsx — New Account Registration

---

SignupScreen.tsx mirrors the Login screen's layout but with three inputs (name, email, password) plus the same social sign-in and “Remember me” options, matching the wireframe's Sign Up card.

## What it does

- **Form state & validation** — captures name, email, and password, with client-side checks (email format, password minimum length/strength) before enabling the SIGN UP button.
- **Submit handler** — calls a React Query mutation (`useSignupMutation`) that POSTs the new-user payload to the FastAPI `/auth/register` endpoint, validated server-side via Pydantic v2 schemas.
- **Auto-login on success** — on a successful registration response, the returned token is stored the same way as in LoginScreen.tsx and the user is taken directly into the Home dashboard, skipping a separate login step.
- **Social sign-up** — Google/Apple buttons perform the same native OAuth exchange as on the Login screen; if the identity is new, the backend creates an account instead of matching an existing one.
- **Duplicate-account handling** — if the email is already registered, the backend returns a conflict error that the screen surfaces inline, along with a shortcut back to Login.
- **Navigation** — the “Already have an account? Login” link routes back to LoginScreen.tsx, completing the Onboarding loop shown in the wireframe.

---

Together, App.tsx, SplashScreen.tsx, LoginScreen.tsx, and SignupScreen.tsx form the complete Onboarding phase: App.tsx sets up the environment, SplashScreen.tsx decides where the user lands, and Login/SignupScreen.tsx handle getting the user authenticated into the Home dashboard.